Experiment 2 : Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

Aim:

Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

1. Dataset Source

Data set link : https://www.kaggle.com/datasets/sathvikisikella/cleaned-air-quality-india
Air Quality Dataset (Environmental Pollution Data)
The dataset contains measurements of different air pollutants collected from monitoring stations.
The dataset is suitable for regression analysis because the target variable PM2.5 concentration is a continuous numerical value.
As per experimental guidelines, a different dataset (Air Quality dataset) is used instead of the Insurance dataset.

2. Dataset Description

This dataset contains environmental attributes that influence air pollution levels, especially PM2.5 concentration.

Input Features (Independent Variables)

PM10 – Particulate matter (10 micrometers)
NO_2 – Nitrogen dioxide concentration
SO_2 – Sulfur dioxide concentration
CO – Carbon monoxide concentration
O_3 – Ozone level
(Other numerical pollutant parameters if present)

Target Variable (Dependent Variable)
PM2.5 – Fine particulate matter concentration (continuous value)

Dataset Characteristics

- Type: Structured tabular dataset

- Number of records: (mention your dataset size here) ● Number of

attributes: (mention number of columns) ● Missing values: Checked and handled during preprocessing ● Learning task: Regression

This dataset is suitable for:

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression

3. Mathematical Formulation of Algorithms

A. Multiple Linear Regression

Multiple Linear Regression predicts PM2.5 levels using multiple pollutant features.

Mathematical Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Where:

- y = predicted PM2.5 value
- x = input pollutant features
- β = regression coefficients

The model minimizes Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

B. Ridge Regression

Ridge Regression adds L2 regularization to reduce overfitting.

$$Loss = MSE + \lambda \sum \beta^2$$

Where:

- λ (alpha) = regularization parameter
- Penalizes large coefficient values

C. Lasso Regression

Lasso Regression adds L1 regularization.

$$Loss = MSE + \lambda \sum |\beta|$$

Benefits:

- Performs feature selection
- Sets some coefficients to zero
- Produces simpler models

4. Algorithm Limitations:

Multiple Linear Regression

- Assumes linear relationship
- Sensitive to outliers
- Cannot handle multicollinearity effectively

Ridge Regression

- Cannot eliminate irrelevant features completely
- Requires tuning of λ

Lasso Regression

- May remove important features if λ is too high
- Unstable when features are highly correlated

5. Methodology / Workflow

Steps:

1. Load Air Quality dataset

2. Select numerical pollutant features

3. Define target variable (PM2.5)

4. Feature scaling using StandardScaler 5.

Train-test split

6. Train models:

    ○ Multiple Linear Regression

    ○ Ridge Regression

    ○ Lasso Regression

7. Evaluate performance using MSE and R^2

8. Perform hyperparameter tuning using GridSearchCV

Workflow Diagram

Dataset → Preprocessing → Feature Scaling → Train-Test Split → Model Training → Prediction → Evaluation

6. Performance Analysis

Evaluation Metrics Used:

- Mean Squared Error (MSE)

- R^2 Score

Interpretation (Write According to Your Output):

- Ridge Regression performed better due to regularization
- Lasso reduced some feature coefficients
- Regularization improved generalization and reduced overfitting

7. Hyperparameter Tuning:

The following hyperparameters were optimized using GridSearchCV:
Ridge Regression
alpha (λ): Controls L2 regularization strength
Lasso Regression
alpha (λ): Controls L1 regularization strength
Best parameters were selected based on highest R^2 score and lowest MSE.
Hyperparameter tuning improved predictive performance and reduced model overfitting.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV from
sklearn.linear_model import LinearRegression, Ridge, Lasso from
sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

sns.set_style("whitegrid")
sns.set_context("talk")

from google.colab import files
print("Upload your CSV dataset")
uploaded = files.upload()

file_name = list(uploaded.keys())[0]

try:
```

```
    df = pd.read_csv(file_name)
except:
    df = pd.read_csv(file_name, sep=';', engine='python')

print("\n ✅Dataset Loaded")
print("Original Shape:", df.shape)

df = df.select_dtypes(include=np.number) df
= df.dropna()

print("After Cleaning Shape:", df.shape)

target_column = df.columns[-1]
print("Target Column:", target_column)

X = df.drop(target_column, axis=1)
y = df[target_column]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y, test_size=0.2, random_state=42 )

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

ridge = Ridge()
param_ridge = {'alpha':[0.001,0.01,0.1,1,10,100]}
grid_ridge = GridSearchCV(ridge, param_ridge, cv=5)
grid_ridge.fit(X_train, y_train)

best_ridge = grid_ridge.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

lasso = Lasso(max_iter=10000)
param_lasso = {'alpha':[0.001,0.01,0.1,1,10]} grid_lasso =
GridSearchCV(lasso, param_lasso, cv=5)
grid_lasso.fit(X_train, y_train)
best_lasso = grid_lasso.best_estimator_
```

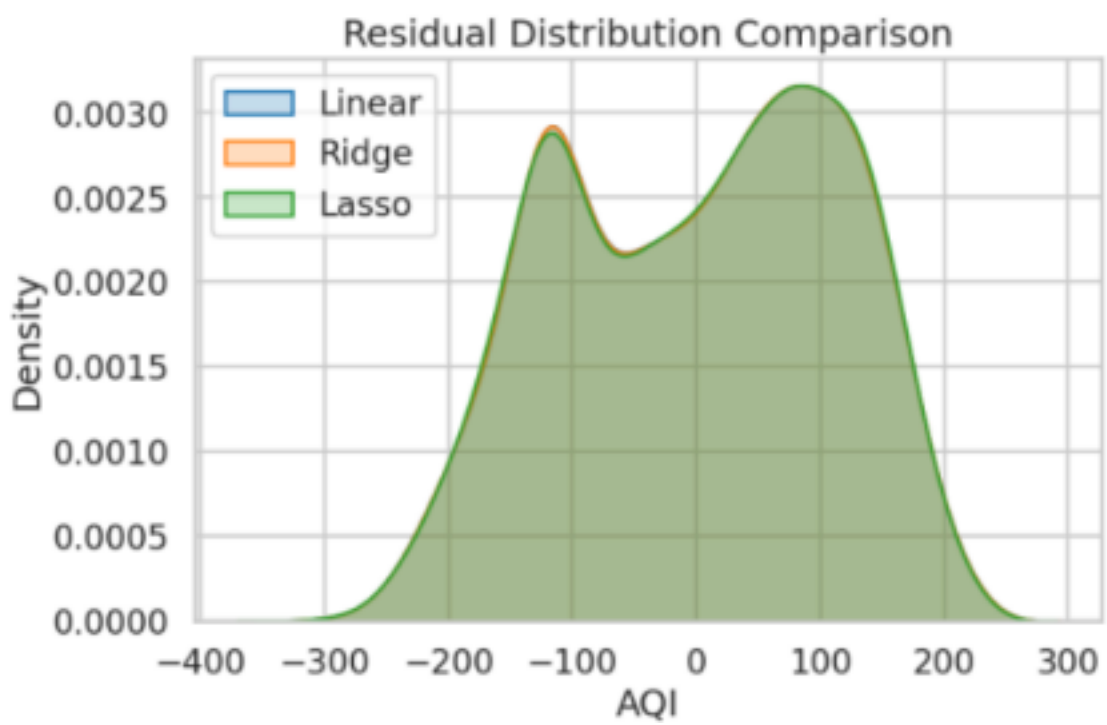
```
y_pred_lasso = best_lasso.predict(X_test)

def evaluate(name, y_true, y_pred):
    print(f"\n{name}")
    print("R2 Score :", r2_score(y_true, y_pred))
    print("MSE :", mean_squared_error(y_true, y_pred)) print("RMSE :",
    np.sqrt(mean_squared_error(y_true, y_pred)))

evaluate("Linear Regression", y_test, y_pred_lr)
evaluate("Best Ridge Regression", y_test, y_pred_ridge)
evaluate("Best Lasso Regression", y_test, y_pred_lasso)

print("\nBest Ridge Alpha:", grid_ridge.best_params_)
print("Best Lasso Alpha:", grid_lasso.best_params_)

models = ["Linear", "Ridge", "Lasso"]
r2_scores = [
    r2_score(y_test, y_pred_lr),
    r2_score(y_test, y_pred_ridge),
    r2_score(y_test, y_pred_lasso)
]

plt.figure(figsize=(8,5))
sns.barplot(x=models, y=r2_scores)
plt.title("R2 Score Comparison")
plt.show()

residual_lr = y_test - y_pred_lr
residual_ridge = y_test - y_pred_ridge
residual_lasso = y_test - y_pred_lasso

plt.figure(figsize=(8,5))
sns.kdeplot(residual_lr, label="Linear", fill=True)
sns.kdeplot(residual_ridge,    label="Ridge",    fill=True)
sns.kdeplot(residual_lasso,    label="Lasso",    fill=True)
plt.title("Residual Distribution Comparison")
plt.legend()
plt.show()

plt.figure(figsize=(7,7))
plt.scatter(y_test, y_pred_ridge)
```

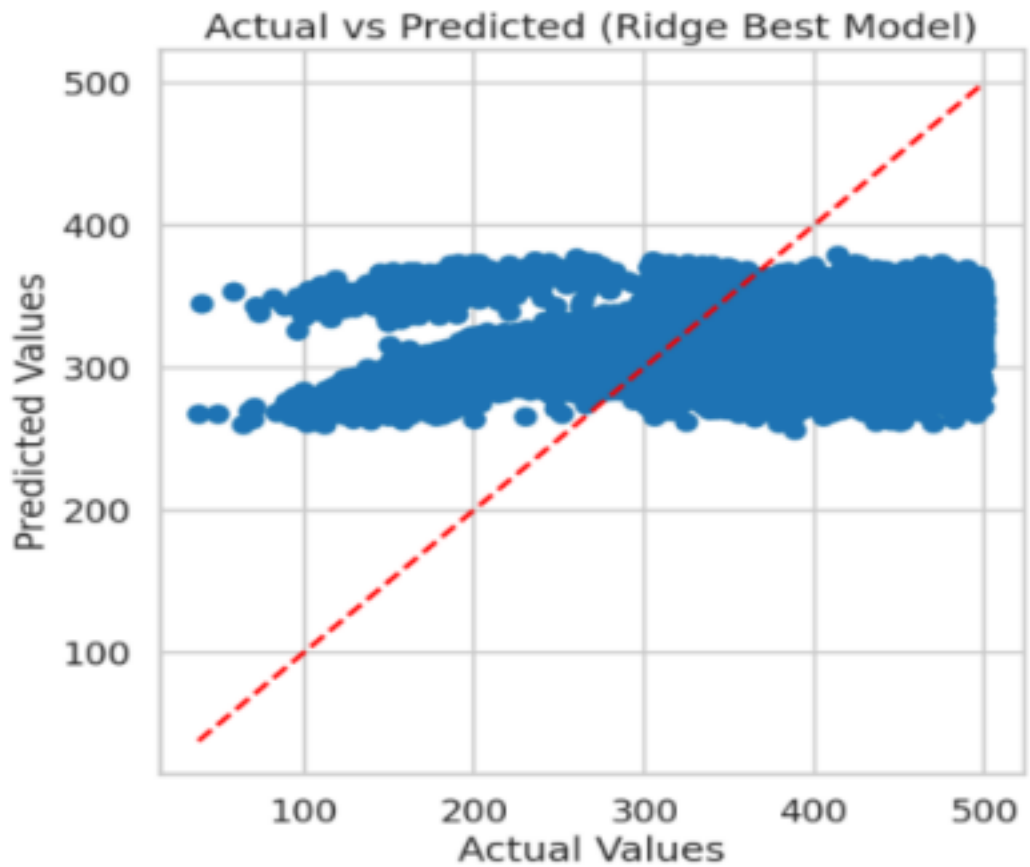
```
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--')
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted (Ridge Best Model)")
plt.show()

errors = pd.DataFrame({
    "Linear": residual_lr,
    "Ridge": residual_ridge,
    "Lasso": residual_lasso
})

plt.figure(figsize=(8,5))
sns.boxplot(data=errors)
plt.title("Error Distribution Comparison")
plt.show()
coef_df = pd.DataFrame({
    "Feature": X.columns,
    "Linear": lr.coef_,
    "Ridge": best_ridge.coef_,
    "Lasso": best_lasso.coef_
})

coef_df.set_index("Feature").plot(kind="bar", figsize=(12,6))
plt.title("Coefficient Comparison")
plt.ylabel("Coefficient Value")
plt.show()

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": np.abs(best_ridge.coef_)
}).sort_values("Importance", ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(data=importance.head(10),
            x="Importance", y="Feature")
plt.title("Top 10 Important Features (Ridge)")
plt.show()
```

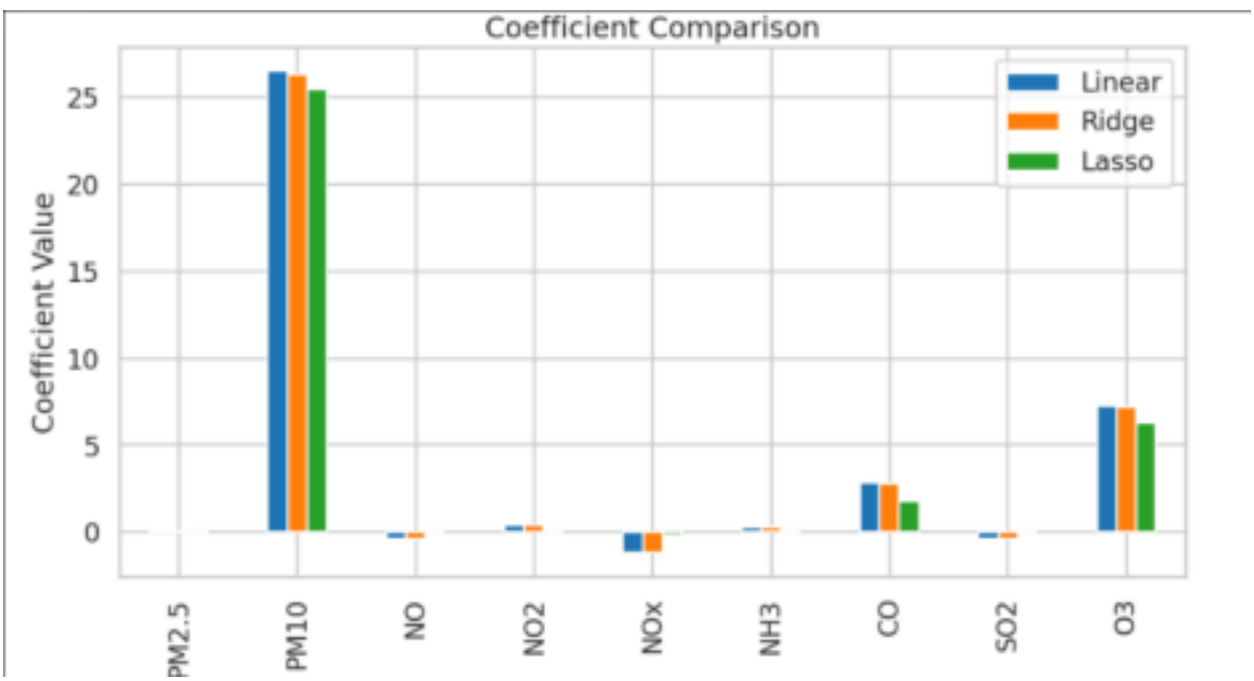
```
alphas = param_ridge['alpha']
ridge_scores = []

for a in alphas:
    model = Ridge(alpha=a)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    ridge_scores.append(r2_score(y_test, pred))

plt.figure(figsize=(8,5))
plt.plot(alphas, ridge_scores, marker='o')
plt.xscale("log")
plt.xlabel("Alpha")
plt.ylabel("R2 Score")
plt.title("Ridge Hyperparameter Tuning Curve")
plt.show()

zero_count = np.sum(best_lasso.coef_ == 0)
print("\nNumber of features eliminated by Lasso:", zero_count)

print("\n ✅Experiment Completed Successfully")
```

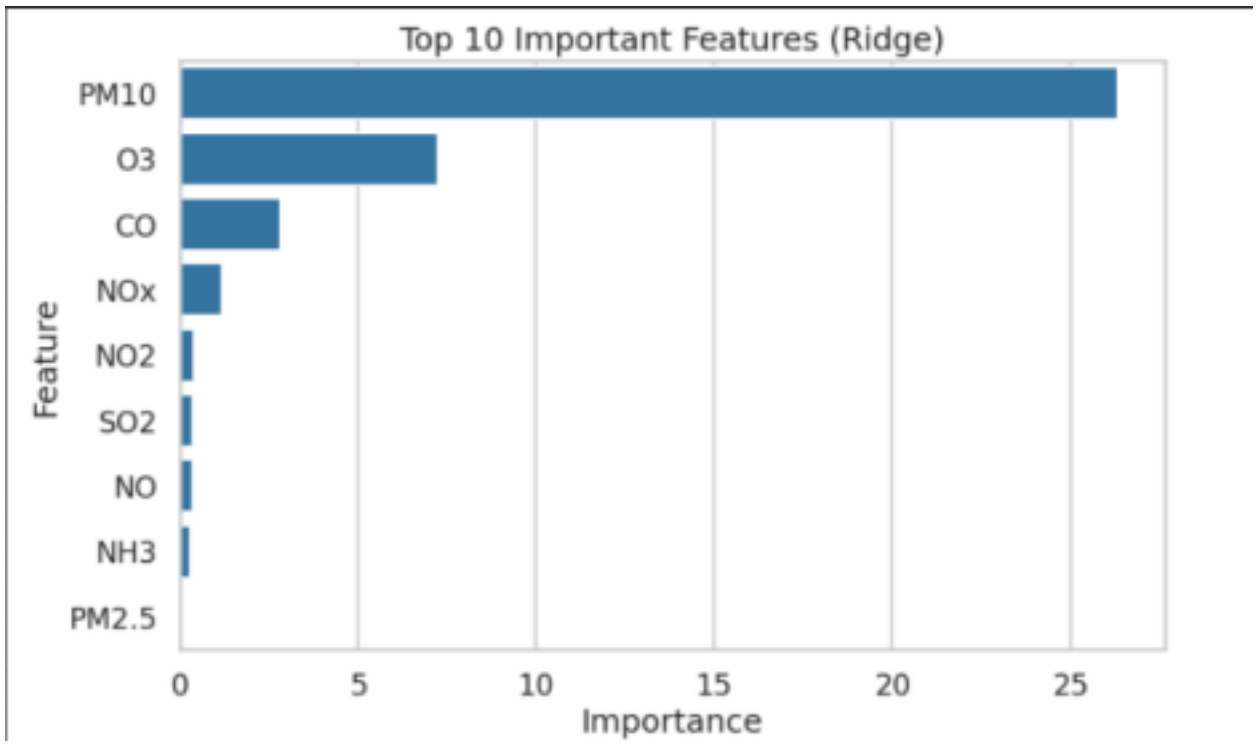
OUTPUT:

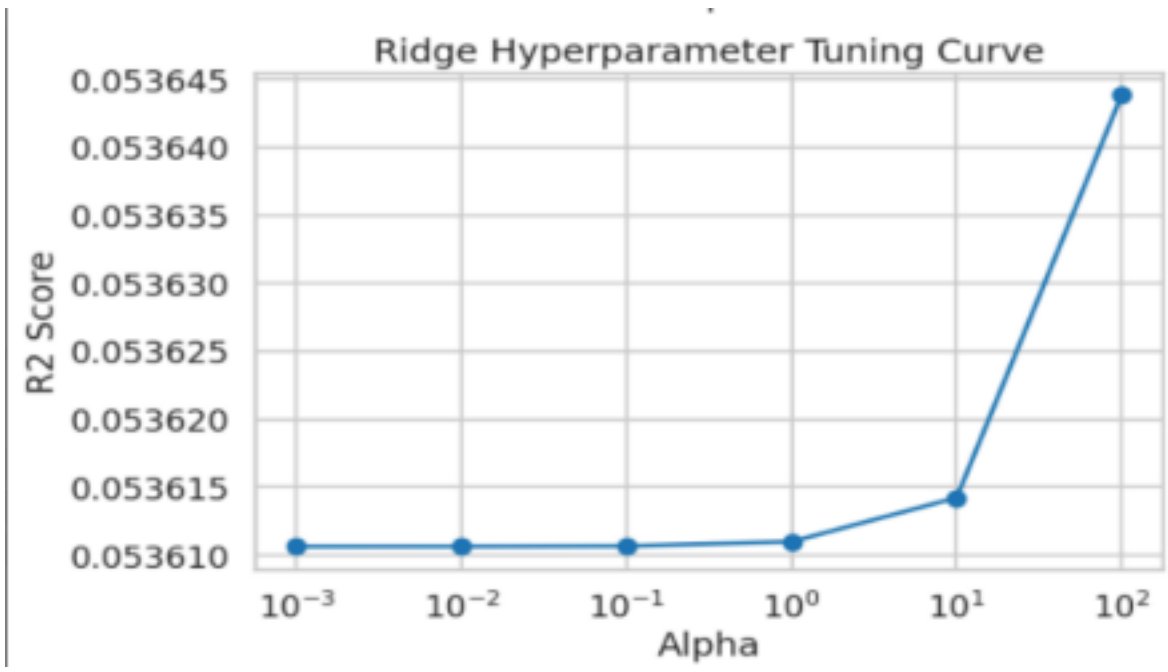
```
Upload your CSV dataset
Choose Files  Air_quality_data.csv
Air_quality_data.csv(text/csv) - 1494181 bytes, last modified: 2/16/2026 - 100% done
Saving Air_quality_data.csv to Air_quality_data (2).csv

✅ Dataset Loaded
Original Shape: (18265, 13)
After Cleaning Shape: (18265, 10)
Target Column: AQI

Linear Regression
R2 Score : 0.053610632587749674
MSE      : 12606.61964732814
RMSE     : 112.27920398421134

Best Ridge Regression
R2 Score : 0.05364384351460827
MSE      : 12606.177252752019
RMSE     : 112.27723390230105

Best Lasso Regression
R2 Score : 0.05337494980178081
MSE      : 12609.759119666312
RMSE     : 112.29318376315774

Best Ridge Alpha: {'alpha': 100}
Best Lasso Alpha: {'alpha': 1}
```




Actual vs Predicted (Ridge Best Model)
Predicted Values
Actual Values
500
400
300
200
100
100
200
300
400
500


Coefficient Comparison
Coefficient Value
Linear
Ridge
Lasso
25
20
15
10
5
0
PM2.5
PM10
NO
NO2
NOx
NH3
CO
SO2
O3





Conclusion:

In this experiment, Linear Regression, Ridge Regression, and Lasso Regression were used to predict PM2.5 levels from air quality data. Ridge and Lasso improved model performance by reducing overfitting through regularization. Among them, Ridge performed slightly better. Overall, regularization techniques enhanced prediction accuracy and model generalization.